# AI-Disrupt PDLC (longread)

Выжимка самого главного · исходник: 25 стр. → 4 стр.

## 01 · Главный сдвиг: от кода к намерению

**AI-Disrupt PDLC переосмысляет создание цифрового продукта: человек определяет смысл и принимает результат, агентная система исполняет, а платформа связывает их работу.**

### Почему одного AI-ассистента недостаточно

Точечная генерация кода ускоряет отдельные операции, но сохраняет согласования, ручную передачу контекста и прежнюю организацию труда. Автор связывает основной эффект с одновременным изменением процессов, ролей и платформы.

### Две петли единого процесса

- **Intent Loop - петля намерения.** Люди формулируют задачу, бизнес-правила, архитектуру и спецификацию; проверяют ценность и принимают результат.
- **Implementation Loop - петля реализации.** Агенты декомпозируют задачу, генерируют код и выполняют автоматические проверки.
- **IDP - интегрированная платформа разработки.** Передаёт намерение в исполнение, сохраняет контекст и возвращает проверяемую обратную связь.

### Человек сохраняет ответственность

Продуктовые и архитектурные решения остаются у человека. В этой версии документа результат агентной цепочки должен быть проверен и принят человеком. Цель - расширить возможности специалистов и убрать рутину.

### Среда исполнения важнее выбора модели

Harness - управляющее окружение модели: инструменты, права, изоляция, восстановление, политики и аудит. Оно превращает вероятностную модель в управляемую корпоративную систему. Инвестиции в такую среду сохраняются при смене моделей.

### Что считать результатом

Автор ставит на первое место клиентскую ценность, доход, прибыль и ускорение процессов, создающих этот результат. Zero-Friction означает устранение технических и организационных препятствий между замыслом и качественным продуктом.

Основа: страницы PDF 2-10. Автор: Кирилл Меньшов; исходник датирован маем 2026 года.

## 02 · Спецификация и корпоративный контекст

**Надёжность агента определяется тем, насколько точно ему переданы цель, ограничения и актуальные знания организации. Спецификация становится главным артефактом разработки.**

### Specification-Driven Development (SDD)

Спецификация описывает целевую функциональность, её смысл, область применения, ограничения, показатели результата, критерии приёмки и риск. Код производен от неё. Исправление начинается с уточнения спецификации, чтобы последующая генерация воспроизводила правильное поведение.

### Context Engineering

Сделайте архитектуру, данные, правила, регламенты и исторические решения доступными модели в машиночитаемом виде. Без контекста агент восполняет пробелы общими знаниями и может действовать вопреки корпоративным требованиям.

### Context Supply Chain

Это процесс создания, проверки, актуализации и доставки контекста. У каждого артефакта должны быть владелец, область применимости, срок актуальности и проверки. Платформа получает данные из мастер-систем, фильтрует и доставляет релевантную часть к моменту решения задачи.

### Три опоры автономной разработки

- **Policy as Code:** обязательные требования исполняются автоматическими правилами и проверками.
- **Цифровой двойник ИТ-систем:** среда для сквозной проверки интеграций и поведения до попадания изменений в промышленную эксплуатацию.
- **ADLC:** отдельный жизненный цикл агентных систем, охватывающий модели, контекст, инструменты, запуск, изоляцию и контроль.

### Принципиальное различие

Документы и инструкции направляют агента. Hooks, ограничения среды и контрольные точки обеспечивают обязательность правил. Критичные требования нельзя оставлять только рекомендациями в тексте.

Основа: страницы PDF 5-6 и 11-16.

## 03 · Платформа, безопасность и расходы

**Платформа должна одновременно давать агентам рабочие инструменты и ограничивать их действия. Контроль качества, доступа и затрат встроен в выполнение задачи.**

### Пять принципов зрелой IDP

- **Agent Runtime / Harness:** управляемый цикл исполнения, инструменты, права, песочница и восстановление сессий.
- **Golden Path:** согласованный маршрут от спецификации до проверенного выпуска.
- **Self-service:** команды расширяют возможности через навыки, API и каталог инструментов.
- **Observability / AgentOps:** журналы, трассировки, потребление токенов и наблюдение за поведением.
- **Security / Governance:** обязательные проверки; всё, что явно не разрешено, требует отдельного разрешения.

### Переиспользование вместо дублирования

Внутренний Agent Marketplace делает проверенные решения доступными другим командам: функции, ограничения, стоимость и опыт использования. Субагенты помогают разделять контекст, права и аудит по специализированным задачам.

### Автономия зависит от риска

Чем выше критичность задачи, чувствительность данных и влияние на промышленную среду, тем больше проверок и человеческих решений. Автор описывает делегирование минорных дефектов до code review с сохранением приёмки исправления инженером.

### Ручное подтверждение не заменяет защиту

Частые запросы разрешений приводят к автоматическому нажатию «да». Основную защиту обеспечивают исполняемые политики и hooks; человеческое согласование сохраняется там, где этого требует риск.

### Agentic FinOps

Учитывайте затраты по задачам, командам и моделям. Задавайте бюджеты, подбирайте модель под сложность, используйте кэширование. Cost Circuit Breakers должны останавливать зацикливание и превышение бюджета с передачей решения человеку.

Основа: страницы PDF 13 и 16-20.

## 04 · Как начать трансформацию

**Начните с измерений и управляемого пилота. Двигайтесь к ясной цели короткими шагами, корректируя следующий шаг по результату предыдущего.**

### Три первых решения из документа

- **Зафиксировать baseline:** текущее состояние продуктивности, качества и стоимости.
- **Запустить пилот:** одна команда из 3-5 инженеров с правом экспериментировать и измеримыми результатами.
- **Назначить AI Champions:** людей, которые накапливают и распространяют практики; это роль проводника изменений.

### Три перекрывающихся горизонта

- **1. Устранение ограничений.** Метрики, контекст и спецификации, базовый Runtime, портал, автоматические проверки и политика использования AI.
- **2. AI-нативные процессы.** SDD для новых продуктов, расширенная IDP, Golden Path, компактные команды и устойчивые сценарии взаимодействия агентов.
- **3. AI-нативная организация.** Большинство рутинных операций выполняют агенты; люди концентрируются на намерении, архитектуре, надзоре и стратегии.

### Как меняются команды и навыки

Tiny Teams в этой версии - 4-5 специалистов с ответственностью за сквозной результат. Развиваются роли Product Engineer, Platform Engineer, AI Architect и другие. Важны постановка намерений, архитектура контекста, системное мышление, AI-грамотность, верификация и ориентация на бизнес.

### Культура как условие внедрения

Нужны короткие циклы обратной связи, проверка гипотез, обучение и право на ошибку. Vibe Working переносит работу через намерение, спецификацию, агентное исполнение и человеческое суждение за пределы разработки - в другие функции компании.

### Как читать количественные обещания

Окно 12-18 месяцев, прогнозы на 2028-2030 годы и показатели роста в исходнике - оценки автора и пересказ отраслевых материалов. Для своей организации эффект подтверждают сравнением с baseline, качеством и бизнес-результатом пилота.

Основа: страницы PDF 10 и 20-25. Выжимка передаёт содержание исходника; внешняя проверка прогнозов не проводилась.

Источник: AI-Disrupt PDLC (longread).pdf. Все ссылки на страницы указывают порядковый номер страницы PDF, включая обложку.